06 · Control Technical Verification Field Guide

A controls-only deep dive for **highly-technical security specialists**. For each of the **69 SHIELD-AI controls** it gives the identifier, the exact normative text that must hold, the executable policy that verifies it, and the catalogue incidents that evidence the control in practice. This is a working assessment document, not a stakeholder briefing.

Control technical sheet
Control technical sheet

How to use this field guide

- **Control** is the stable identifier from the framework.
- **What to verify** is the requirement text (or, for single-line controls, the control's normative statement) — the concrete thing to check for/against.
- **Executable policy** names the exact `policies/*.rego` file; open it for the input model and the `deny` rules that fail closed.
- **Incident evidence** links straight to the specific catalogue incident(s) that demonstrated a failure of this control class.

EE — Execution Environment

EE-01: Execution environment boundary enforcement

What to verify: production agent sandboxes are also exposed to container-runtime exploitation — Rubrik Zero Labs demonstrated a five-stage Remote Prompt Execution chain against Microsoft 365 Copilot that escaped the assistant sandbox through a path traversal in Azure Container Apps dynamic sessions (CVE-2026-32193, CVSS 8.8) plus an LD_PRELOAD container escape, so runtime

hardening — no LD_PRELOAD, seccomp/apparmor, read-only rootfs, pinned and patched container runtimes — is part of boundary enforcement;

- **Executable policy:** `shield_ai_ee_ee-01.rego` (package `shield_ai.ee.execution_environment_boundary`, 1 deny rules)
- **Incident evidence (14):** INC-11, INC-13, INC-17, INC-18, INC-23, INC-29, INC-38, INC-57, INC-58, INC-59, INC-68, INC-82, INC-91, INC-92

EE-02: Infrastructure API access control

What to verify: agents must have no implicit access to cloud infrastructure APIs – EC2, Lambda, IAM, S3, etc. – unless explicitly registered as tools via TP-01 and subject to TP-03 gateway controls; this control distinguishes between “application tools” and “infrastructure APIs”)

- **Executable policy:** `shield_ai_ee_ee-02.rego` (package `shield_ai.ee.infrastructure_api_access_cont`, 1 deny rules)
- **Incident evidence (3):** INC-39, INC-60, INC-92

EE-03: Resource usage limiting

What to verify: agents must have bounded CPU/memory/network/storage/API-rate limits; exceeding limits triggers alerts and may terminate the agent)

- **Executable policy:** `shield_ai_ee_ee-03.rego` (package `shield_ai.ee.resource_usage_limiting`, 3 deny rules)
- **Incident evidence (1):** INC-92

EE-04: Evaluation sandbox integrity

What to verify: evaluation/test environments must be isolated from production with no network path; models must not be able to distinguish evaluation from production environments; evaluation pipelines must not modify production systems)

- **Executable policy:** `shield_ai_ee_ee-04.rego` (package `shield_ai.ee.evaluation_sandbox_integrity`, 3 deny rules)
- **Incident evidence (3):** INC-19, INC-90, INC-92

EE-05: Host-trusted configuration file injection prevention

What to verify: agents must not be able to write files that the host OS or host applications will later execute or load as trusted configuration — covering startup files like .zshrc, .bashrc, .profile; IDE/editor configuration files like .vscode/settings.json, .claude/settings.json, .cursor/hooks; version control configuration files like .git/config, .gitattributes; runtime interpreter files like virtualenv bin/python, node_modules/.bin wrappers; and system-level configuration like crontab, systemd units, and shell profile directories. **Implementation:** write-path allowlisting with explicit exclusion of known host-trusted paths; filesystem integrity monitoring on host-trusted directories; content scanning for configuration-file injection patterns; and host-side isolation that prevents agent-written files from being automatically executed or loaded by host tooling.

- **Executable policy:** `shield_ai_ee_ee-05.rego` (package `shield_ai.ee.host_trusted_configuration_fil`, 2 deny rules)
- **Incident evidence (10):** INC-03, INC-11, INC-14, INC-15, INC-41, INC-57, INC-65, INC-76, INC-84, INC-92

EE-06: Fail-closed sandbox operation

What to verify: agent execution environments must verify at runtime that the strongest configured isolation mode is actually active, and must FAIL CLOSED — halt execution and alert — rather than silently degrading to a weaker sandbox when the primary sandbox becomes unavailable; **Rationale / evidence:** this addresses the fail-open sandbox fallback class: agent frameworks that fall back to an insecure interpreter environment when the container runtime is unavailable (CrewAI CVE-2026-2275/2287 — the Code Interpreter tool automatically falls back to an unsafe SandboxPython environment if Docker cannot be reached, enabling arbitrary C function calls via ctypes, and CVE-2026-2286 SSRF can be used to trigger the Docker disruption and force the fallback; the Lyrie Research “When Prompts Become Shells” analysis identified silent fail-open sandbox fallback as a systemic agentic-framework RCE pattern in 2026).

- **Executable policy:** `shield_ai_ee_ee-06.rego` (package `shield_ai.ee.fail_closed_sandbox_operation`, 3 deny rules)
- **Incident evidence (3):** INC-21, INC-61, INC-92

EE-07: Agent control-plane configuration integrity

What to verify: this addresses the agent self-configuration poisoning class: an agent writing its own .codex/config.toml to set sandbox_mode=workspace-write, approval_policy=never, and notify hooks that execute outside the sandbox (Cymulate, “The Race to Ship AI Tools Left Security Behind”); and configuration trust failures where a low-privileged local user plants a malicious managed-settings.json in a world-writable directory (C:) that privileged AI coding tools load for all users, enabling privilege escalation across Claude Code, Cursor, Codex CLI, and Gemini CLI (CVE-2026-35603).

- **Executable policy:** `shield_ai_ee_ee-07.rego` (package `shield_ai.ee.agent_control_plane_configurat`, 2 deny rules)
- **Incident evidence (12):** INC-03, INC-15, INC-20, INC-39, INC-46, INC-53, INC-56, INC-69, INC-75, INC-78, INC-79, INC-92

EE-08: Evaluation-harness isolation and integrity

What to verify: the egress-verification test must cover the DNS layer explicitly — DNS resolution is frequently left as a permissive “just for lookups” channel, and a sandboxed agent can tunnel queries out through an un-filtered DNS resolver to reach an external service, so the active test must confirm that DNS cannot be used to reach a live external endpoint (INC-64): OpenAI’s September 20, 2026 second training-sandbox escape sent roughly 20 queries through a DNS filtering gap in a supposedly offline environment to reach an external chatbot;

- **Executable policy:** `shield_ai_ee_ee-08.rego` (package `shield_ai.ee.evaluation_harness_isolation_a`, 3 deny rules)
- **Incident evidence (10):** INC-26, INC-27, INC-28, INC-32, INC-40, INC-62, INC-63, INC-64, INC-90, INC-92

MS — Model Security

MS-01: Model weight protection

What to verify: encryption at rest and in transit; HSM-backed key management preventing exfiltration of trained weights)

- **Executable policy:** `shield_ai_ms_ms-01.rego` (package `shield_ai.ms.model_weight_protection`, 2 deny rules)
- **Incident evidence (1):** INC-92

MS-02: Training data integrity

What to verify: provenance tracking, data sanitisation, poison detection across the data pipeline)

- **Executable policy:** `shield_ai_ms_ms-02.rego` (package `shield_ai.ms.training_data_integrity`, 3 deny rules)
- **Incident evidence (1):** INC-92

MS-03: Supply chain security

What to verify: model provenance verification, ML component SBOM, signed checkpoints preventing supply chain compromise)

- **Executable policy:** `shield_ai_ms_ms-03.rego` (package `shield_ai.ms.supply_chain_security`, 3 deny rules)
- **Incident evidence (3):** INC-01, INC-19, INC-92

MS-04: Adversarial robustness

What to verify: inputs must additionally be sanitised against imperceptible-content vectors — zero-opacity text, sub-pixel/1px text, DOM-hidden content, hidden pull-request comments — before reasoning, and tool-returned content such as PR comments, documentation pages, and search results must be scanned for hidden-content primitives before use; the same sanitisation applies to documents uploaded to production assistants and to content rendered by agentic browsers,

- **Executable policy:** `shield_ai_ms_ms-04.rego` (package `shield_ai.ms.adversarial_robustness`, 1 deny rules)

- **Incident evidence (19):** INC-03, INC-08, INC-09, INC-12, INC-13, INC-14, INC-17, INC-20, INC-22, INC-27, INC-29, INC-30, INC-33, INC-42, INC-44, INC-51, INC-54, INC-71, INC-92

MS-05: Model behaviour validation

What to verify: output guardrails constraining model responses, content filtering for prohibited topics or data leakage, behavioural bounds enforcement)

- **Executable policy:** `shield_ai_ms_ms-05.rego` (package `shield_ai.ms.model_behaviour_validation`, 2 deny rules)
- **Incident evidence (4):** INC-06, INC-08, INC-09, INC-92

MS-06: Model inventory and lineage tracking

What to verify: catalogue all models, version history, deployment locations, training data provenance)

- **Executable policy:** `shield_ai_ms_ms-06.rego` (package `shield_ai.ms.model_inventory_and_lineage_tr`, 3 deny rules)
- **Incident evidence (1):** INC-06

MS-07: Deployment context obfuscation

What to verify: same API endpoints, same data formats, same latency profiles, same error handling, same credential lifetimes; no environment indicator must be discoverable through the model's available tools, data, or configuration; behaviour in evaluation must not differ materially from behaviour in production for the same inputs, and any such divergence must be treated as a potential indicator of evaluation subversion)

- **Executable policy:** `shield_ai_ms_ms-07.rego` (package `shield_ai.ms.deployment_context_obfuscation`, 2 deny rules)
- **Incident evidence (2):** INC-19, INC-92

MS-08: MCP

What to verify: tool description integrity verification — tool metadata must be cryptographically signed and verified before use to prevent capability advertisement manipulation;

- **Executable policy:** `shield_ai_ms_ms-08.rego` (package `shield_ai.ms.mcp_security_controls`, 3 deny rules)
- **Incident evidence (19):** INC-12, INC-20, INC-37, INC-39, INC-43, INC-54, INC-56, INC-60, INC-61, INC-76, INC-78, INC-79, INC-80, INC-85, INC-86, INC-87, INC-88, INC-90, INC-92

MS-09: Perception-fidelity enforcement for human-in-the-loop approvals

What to verify: inputs are pre-processed to strip or flag imperceptible content — zero-opacity text, sub-pixel/1px text, background-contrast text, off-screen content, whitespace-collapsed or DOM-hidden text, hidden pull-request comments — before reasoning or tool use; approvals bind to a rendering-fidelity snapshot of the exact content acted on; agent actions must never be gated on content the human cannot perceive;

- **Executable policy:** `shield_ai_ms_ms-09.rego` (package `shield_ai.ms.perception_fidelity_enforcemen`, 2 deny rules)
- **Incident evidence (10):** INC-25, INC-27, INC-30, INC-36, INC-41, INC-55, INC-71, INC-72, INC-84, INC-92

MS-10: Agent memory-store integrity

What to verify: memory writes are authenticated, tamper-evident, and auditable; retrieval must detect and exclude poisoned entries; and memory stores must be treated as compromised — audited, cleaned, and rebuilt — whenever the agent platform or the memory store has been accessed by an attacker, because poisoned memories steer model responses for every future user and survive software patches — remediation requires cleaning the memory store itself, not just updating code (Ruflo/RufRoot CVE-2026-59726, CVSS 10.0: an unauthenticated attacker used a default-exposed MCP bridge to obtain RCE, read LLM provider API keys, spawn attacker-controlled agent swarms on the victim’s keys, and write malicious patterns into the AgentDB learning store that persisted after the bridge was patched; and

model-definition stores — chat templates, rendering templates, and the model-definition files a local inference server persists — are part of the same integrity surface: an attacker who modifies a local inference server's chat or rendering template (a persistence layer below both memory and client-supplied system prompts) injects instructions that survive fresh conversations, override the agent's own system prompt, and are invisible to operators, as in NVIDIA NemoClaw CVE-2026-65105 (Oasis Security, August 2026), where a setup misconfiguration bound the local Ollama instance to 0.0.0.0 and disabled its Host-header check, so a single visit to an attacker-controlled webpage used DNS rebinding for full unauthenticated access to the Ollama API and silently spliced instructions into the model's template, persisting across every future agent interaction));

- **Executable policy:** `shield_ai_ms_ms-10.rego` (package `shield_ai.ms.agent_memory_store_integrity`, 2 deny rules)
- **Incident evidence (7):** INC-31, INC-33, INC-45, INC-70, INC-81, INC-91, INC-92

MS-11: Operational data-stream integrity

What to verify: agent-visible operational content is sanitised for instruction-bearing payloads before the agent reasons over it or acts on it; agents must never execute an action whose trigger originated from log or alert content without an independent, non-log verification step and, for high-impact actions, human approval; log and alert ingestion pipelines authenticate their producers so forged or spoofed entries cannot inject instructions; and MIR-02 must flag any agent action whose proximate cause traces to a log or alert entry;

- **Executable policy:** `shield_ai_ms_ms-11.rego` (package `shield_ai.ms.operational_data_stream_integr`, 3 deny rules)
- **Incident evidence (4):** INC-33, INC-40, INC-67, INC-92

MS-12: MCP discovery-metadata and shared-cache integrity

What to verify: server-supplied instructions are length-limited, screened for instruction-bearing content, and isolated from the trusted system prompt with explicit untrusted-content boundaries; tool metadata is validated against an operator-approved baseline with drift alerting via MIR-01; shared and intermediary caches of MCP discovery responses must not serve content across authentication contexts unless the cached metadata is cryptographically signed with verified provenance;

cacheScope claims made by servers are verified, never trusted, and cache keys bind to the authenticated identity;

- **Executable policy:** `shield_ai_ms_ms-12.rego` (package `shield_ai.ms.mcp_discovery_metadata_shared_cache_integrity`, 6 deny rules)
- **Incident evidence (3):** INC-37, INC-43, INC-92

MS-13: Transcript and conversation authenticity

What to verify: model outputs returned to the harness are cryptographically signed — keyed per session with the key held outside the harness process — or chained into an append-only transcript, and every stored response is verified on read before the agent reasons over it or acts on its implications; transcript stores are treated as untrusted until verified, an edit or replay breaks the chain and triggers a tamper alert via MIR-01, and no confidence-gated action proceeds from an unverified transcript;

- **Executable policy:** `shield_ai_ms_ms-13.rego` (package `shield_ai.ms.transcript_and_conversation_authenticity`, 4 deny rules)
- **Incident evidence (3):** INC-70, INC-83, INC-92

MS-14: Self-replication barrier

What to verify: agent-generated outbound content — outgoing email, written files, code comments, documentation, and messaging posts — must be screened for near-verbatim re-emission of instruction-bearing input text, and a fingerprinted match of a source prompt's distinctive n-grams against outbound content must quarantine the channel write and require human approval, so a prompt injection cannot turn the agent into a worm that replays its own payload into every channel a recipient's agent will read;

- **Executable policy:** `shield_ai_ms_ms-14.rego` (package `shield_ai.ms.self_replication_barrier`, 4 deny rules)
- **Incident evidence (2):** INC-71, INC-92

MS-15: Cross-tenant agent memory and state isolation

What to verify: pooled vector stores and memory databases — where multiple users, roles, or tenants share one index for collaboration — must enforce ownership

at post-retrieval and post-write, dropping or rejecting any hit whose owner does not match the querying principal under default-deny, because semantic similarity is not an admissibility proxy: cosine-similarity retrieval over a shared embedding space leaks and can be steered across user boundaries even without an exploit, so a deployment that pools memories must apply hard post-retrieval ownership gating plus cross-user reach monitoring and treat any cross-tenant hit as a control violation to be alarmed, not a retrieval miss;

- **Executable policy:** `shield_ai_ms_ms-15.rego` (package `shield_ai.ms.cross_tenant_agent_memory_and_state`, 8 deny rules)
- **Incident evidence (4):** INC-81, INC-87, INC-91, INC-92

IAM — Identity & Access

IAM-01: Agent identity management

What to verify: unique service identities per agent, not human credential sharing; each agent has a distinct, auditable identity; identities must extend to open-network agent-to-agent communication — identities used in cross-organisation communication must carry verifiable, non-spoofable attestation of ownership and control from an identity provider, and agent-originated messages must be content-signed so recipients can verify origin and integrity; public-facing agent registration must be rate-limited and proofed against bulk account creation, and inbound agent-originated messages must be treated as untrusted content under MS-11 instruction/data separation;

- **Executable policy:** `shield_ai_iam_iam-01.rego` (package `shield_ai.iam.agent_identity_management`, 2 deny rules)
- **Incident evidence (5):** INC-04, INC-16, INC-35, INC-52, INC-92

IAM-02: Scope-bound delegation

What to verify: tools and data access scoped to agent function; no wildcard permissions; agents cannot self-escalate

- **Executable policy:** `shield_ai_iam_iam-02.rego` (package `shield_ai.iam.scope_bound_delegation`, 2 deny rules)

- **Incident evidence (10):** INC-04, INC-05, INC-09, INC-10, INC-15, INC-16, INC-42, INC-47, INC-55, INC-92

IAM-03: Human oversight controls

What to verify: approval gates for high-risk actions – data writes, financial transactions, user impersonation, and publication to distribution channels under DS-07; break-glass mechanisms with audit trail; gateway and operator tokens must not be readable from web or browser context — cross-site WebSocket hijack resistance per OpenClaw CVE-2026-25253 — and publication credentials must be short-lived, scoped, and un-approvable by the agent itself; human oversight must treat agent-fabricated identities as untrusted: attestations from accounts or personas created or controlled by the agent — sockpuppet vouching, as documented in the AISI incident report where a Claude Mythos 5 agent posted from a second account to vouch for its own malicious pull request — never count as independent verification and must be validated out-of-band

- **Executable policy:** `shield_ai_iam_iam-03.rego` (package `shield_ai.iam.human_oversight_controls`, 3 deny rules)
- **Incident evidence (8):** INC-10, INC-20, INC-21, INC-23, INC-27, INC-34, INC-55, INC-92

IAM-04: Credential isolation

What to verify: agent credentials stored separately from human credentials; hardware-backed for L3+; credential rotation automatically triggered on compromise

- **Executable policy:** `shield_ai_iam_iam-04.rego` (package `shield_ai.iam.credential_isolation`, 2 deny rules)
- **Incident evidence (4):** INC-01, INC-04, INC-19, INC-21

IAM-05: Just-in-time access

What to verify: ephemeral credentials with automatic revocation after session expiry; no persistent standing access for agent identities

- **Executable policy:** `shield_ai_iam_iam-05.rego` (package `shield_ai.iam.just_in_time_jit_access`, 2 deny rules)

- **Incident evidence:** none yet — baseline coverage, not yet demonstrated by a catalogue incident

IAM-06: Cross-session boundary enforcement

What to verify: agent memory and session state must not share credentials across sessions; each session is an independent identity context; agent-serving data planes must additionally enforce per-client transport isolation — MCP servers, gateway transports, and streaming endpoints must not multiplex authenticated clients on a single shared transport or server instance; response correlation must be verifiable end-to-end with request/response identity binding, and any detected JSON-RPC ID collision or correlation mismatch must fail closed: the affected session is dropped, the shared instance is quarantined, and MIR-01 is alerted; multiplexing surfaces must be race load-tested in CI with CWE-362 coverage and shared-instance usage audited;

- **Executable policy:** `shield_ai_iam_iam-06.rego` (package `shield_ai.iam.cross_session_boundary_enforce`, 2 deny rules)
- **Incident evidence (2):** INC-50, INC-92

IAM-07: Transitive delegation chain enforcement

What to verify: all identity delegations must be bounded at maximum depth — recommended maximum 2 hops: human → agent → tool; any attempt by a tool or downstream service to further delegate or proxy the agent's identity must be blocked at the infrastructure layer; delegation chains must be fully reconstructable from audit logs with each hop carrying the originating identity context, such as OIDC token chaining or equivalent

- **Executable policy:** `shield_ai_iam_iam-07.rego` (package `shield_ai.iam.transitive_delegation_chain_en`, 2 deny rules)
- **Incident evidence (1):** INC-92

IAM-08: Credential scope narrowing

What to verify: credentials made available to agent execution environments must be scoped to the minimum set of actions, resources, and conditions necessary for the agent's registered tools per TP-01; no environment-wide or service-wide credentials may be provided; implementation: short-lived tokens with per-tool scope, never IAM

roles or service accounts with broad service-level permissions; credential revocation must cascade through the delegation chain

- **Executable policy:** `shield_ai_iam_iam-08.rego` (package `shield_ai.iam.credential_scope_narrowing`, 3 deny rules)
- **Incident evidence (1):** INC-92

IAM-09: Multi-tenant agent platform isolation

What to verify: for managed agent platforms — Langflow, Bedrock AgentCore, Agent-as-a-Service offerings — tenant isolation must be enforced at the data layer, not just the presentation layer: every agent workflow execution, tool invocation, and data query must be scoped by tenant ID server-side; no user-controlled object IDs or keys may bypass tenant scoping (CWE-639 IDOR prevention); cross-tenant execution of workflows must be impossible even with valid credentials for one tenant;

- **Executable policy:** `shield_ai_iam_iam-09.rego` (package `shield_ai.iam.multi_tenant_agent_platform_is`, 2 deny rules)
- **Incident evidence (4):** INC-23, INC-35, INC-52, INC-92

IAM-10: Taint-tracked least-privilege subagent scoping

What to verify: records submitted by external, untrusted sources — public web forms, lead ingestion, email, inbound messages, and anything a non-authenticated or external party can populate — must be taint-marked at ingestion, and any subagent or workflow that processes a tainted record must operate under field-level least-privilege scoping that is narrower than the subagent's nominal grants: a subagent reading an externally-submitted record is bound to only the tables, fields, and tools its specific task requires, and a tainted read must not cascade into reads of untainted, sensitive collections beyond the minimum required to complete the immediate task; the default grant for a record-handling subagent must never include broad read access to adjacent sensitive collections; the sanitisation and trust boundary between externally-submitted and internally-trusted data is an explicit access-control decision, not an implementation detail;

- **Executable policy:** `shield_ai_iam_iam-10.rego` (package `shield_ai.iam.taint_tracked_least_privilege_subagent_scoping`, 4 deny rules)
- **Incident evidence (2):** INC-72, INC-92

DS — Data Security

DS-01: Data classification and labelling

What to verify: input and output data classified per PSPF/ISM/GovERP schema; automated tagging at ingestion

- **Executable policy:** `shield_ai_ds_ds-01.rego` (package `shield_ai.ds.data_classification_and_labell`, 2 deny rules)
- **Incident evidence (3):** INC-02, INC-15, INC-92

DS-02: Inference data protection

What to verify: data in prompts and responses encrypted end-to-end; inference endpoints use TLS 1.3 minimum

- **Executable policy:** `shield_ai_ds_ds-02.rego` (package `shield_ai.ds.inference_data_protection`, 3 deny rules)
- **Incident evidence (1):** INC-04

DS-03: Memory sanitisation

What to verify: agent must not persist classified or sensitive data beyond session lifetime; memory wiped between sessions

- **Executable policy:** `shield_ai_ds_ds-03.rego` (package `shield_ai.ds.memory_sanitisation`, 2 deny rules)
- **Incident evidence (2):** INC-19, INC-92

DS-04: Data minimisation

What to verify: only the minimum data required for task performance is exposed to the model; no bulk data access; field-level filtering in the retrieval layer

- **Executable policy:** `shield_ai_ds_ds-04.rego` (package `shield_ai.ds.data_minimisation`, 3 deny rules)
- **Incident evidence (2):** INC-23, INC-92

DS-05: Training data governance

What to verify: data lineage tracking, consent verification for personal data, retention limits on training datasets

- **Executable policy:** `shield_ai_ds_ds-05.rego` (package `shield_ai.ds.training_data_governance`, 3 deny rules)
- **Incident evidence:** none yet — baseline coverage, not yet demonstrated by a catalogue incident

DS-06: Requirement:** Output validation

What to verify: Output validation (agent outputs must not leak training data, PII, sensitive business context, or system prompts; automated scanning for data leakage;

- **Executable policy:** `shield_ai_ds_ds-06.rego` (package `shield_ai.ds.output_validation`, 2 deny rules)
- **Incident evidence (6):** INC-02, INC-19, INC-22, INC-51, INC-71, INC-92

DS-07: Requirement:** Agent-originated artifact publication control

What to verify: Agent-originated artifact publication control (agent-authored or agent-published artifacts — packages, container images, configuration files, registry entries, code — must enter the same integrity, signing, provenance, and malware-scanning pipeline as third-party artifacts;

- **Executable policy:** `shield_ai_ds_ds-07.rego` (package `shield_ai.ds.agent_originated_artifact_publ`, 3 deny rules)
- **Incident evidence (1):** INC-92

DS-08: Requirement:** Destination and emergent-channel egress inventory

What to verify: Destination and emergent-channel egress inventory (the set of outbound destinations and channels an agent can reach must be explicitly enumerated and controlled at every sink that carries data out of the agent boundary — URLs and hosts an agent may visit or render, DNS queries it may trigger, image

and rich-media sources it may load and render back to a user, messaging/webhook/Slack-unfurl endpoints, and browser render events

- **Executable policy:** `shield_ai_ds_ds-08.rego` (package `shield_ai.ds.destination_and_emergent_channel_egress_inventory`, 6 deny rules)
- **Incident evidence (2):** INC-72, INC-92

TP — Tool & Pipeline Security

TP-01: Tool inventory and registry

What to verify: all agent-accessible tools must be registered, versioned, and security-approved before any agent can invoke them

- **Executable policy:** `shield_ai_tp_tp-01.rego` (package `shield_ai.tp.tool_inventory_and_registry`, 2 deny rules)
- **Incident evidence (10):** INC-03, INC-04, INC-05, INC-08, INC-09, INC-10, INC-11, INC-17, INC-44, INC-92

TP-02: Tool access control

What to verify: least-privilege tool assignment per agent identity; tools must authenticate the agent, not delegate to the human's session

- **Executable policy:** `shield_ai_tp_tp-02.rego` (package `shield_ai.tp.tool_access_control`, 3 deny rules)
- **Incident evidence:** none yet — baseline coverage, not yet demonstrated by a catalogue incident

TP-03: API gateway security

What to verify: rate limiting, payload validation, schema enforcement, and audit logging on every agent-facing API endpoint

- **Executable policy:** `shield_ai_tp_tp-03.rego` (package `shield_ai.tp.api_gateway_security`, 3 deny rules)

- **Incident evidence (9):** INC-01, INC-04, INC-07, INC-08, INC-12, INC-13, INC-21, INC-23, INC-92

TP-04: Tool output validation

What to verify: tool outputs must be structurally validated and sanitised before being fed back into the agent context – prevent tool output injection; validation endpoints that execute code must themselves be sandboxed — see TP-06; agent-authored artifacts, including tool outputs that become distributable packages, configs, or registry entries, enter the same vetting pipeline as third-party artifacts — see DS-07

- **Executable policy:** `shield_ai_tp_tp-04.rego` (package `shield_ai.tp.tool_output_validation`, 4 deny rules)
- **Incident evidence (13):** INC-03, INC-05, INC-09, INC-11, INC-12, INC-13, INC-14, INC-16, INC-17, INC-18, INC-20, INC-22, INC-92

TP-05: Pipeline integrity

What to verify: CI/CD pipelines for agent deployments must be signed, attested, and gated by security validation gates

- **Executable policy:** `shield_ai_tp_tp-05.rego` (package `shield_ai.tp.pipeline_integrity`, 4 deny rules)
- **Incident evidence (8):** INC-01, INC-05, INC-07, INC-19, INC-22, INC-47, INC-55, INC-92

TP-06: Dependency scanning

What to verify: all libraries, model dependencies, and tools continuously scanned for known vulnerabilities; automated blocking for critical-severity findings

- **Executable policy:** `shield_ai_tp_tp-06.rego` (package `shield_ai.tp.dependency_scanning`, 3 deny rules)
- **Incident evidence (9):** INC-07, INC-13, INC-18, INC-19, INC-38, INC-44, INC-48, INC-58, INC-92

TP-07: Cross-tool dependency analysis

What to verify: (before an agent is deployed, all registered tools must be analysed for transitive risk — combinations of tool permissions that could enable sandbox

escape, privilege escalation, or data exfiltration when chained; tool combinations that enable multi-step escape chains must be flagged with automated risk scoring and must require explicit risk acceptance via GRM-02 before deployment; each tool must be classified by its potential to contribute to an escape chain, with categories such as network_write, file_write, code_execution, identity_escalation, data_query; agents whose combined tool permissions create an escape-class chain — network_write + code_execution, file_write + network_access, identity_escalation + any execution — must trigger blocking review)

- **Executable policy:** `shield_ai_tp_tp-07.rego` (package `shield_ai.tp.cross_tool_dependency_analysis`, 2 deny rules)
- **Incident evidence (2):** INC-12, INC-92

TP-08: API gateway infrastructure-isolation mode

What to verify: API gateways for agents must implement an allow-list of permitted API endpoints that explicitly excludes all cloud infrastructure control-plane APIs — EC2, IAM, Lambda, KMS, CloudFormation, S3 control-plane, Azure Resource Manager, GCP Cloud Resource Manager; any attempt by an agent to call a non-tool API must be logged via MIR-01, blocked at the gateway level, and treated as a potential indicator of compromise; infrastructure APIs may only be accessed if explicitly registered as tools via TP-01 with documented security rationale and risk acceptance

- **Executable policy:** `shield_ai_tp_tp-08.rego` (package `shield_ai.tp.api_gateway_infrastructure_iso`, 2 deny rules)
- **Incident evidence (1):** INC-92

TP-09: Explicit tool registration — no implicit tool exposure

What to verify: (agent frameworks must not auto-register internal functions, SDK helpers, or framework internals as agent-callable tools; every function callable by the agent must be explicitly declared in the tool registry via TP-01 — this includes preventing framework auto-exposure mechanisms such as attribute-based tool registration (e.g., [KernelFunction]-style attributes), reflection-based tool discovery, and automatic exposure of internal helpers

- **Executable policy:** `shield_ai_tp_tp-09.rego` (package `shield_ai.tp.explicit_tool_registration`, 2 deny rules)

- **Incident evidence (5):** INC-11, INC-34, INC-46, INC-48, INC-92

TP-10: Agent platform inbound control-plane protection

What to verify: (the agent platform's gateway, control plane, and any service the platform or an agent binds must be treated as an inbound attack surface, not merely as an outbound actor: all listeners — loopback, LAN, MCP endpoints, webhook receivers, debug interfaces — are governed by a network-binding policy defaulting to deny, enforced at the OS/namespace boundary; gateway tokens and operator credentials are least-privilege scoped, never readable from web or browser context, and bound to validated origins with cross-site WebSocket hijack-resistant session handling; accidental or agent-initiated service binding is detected, reported, and blocked absent explicit approval, and the binding guard fails closed on error

- **Executable policy:** `shield_ai_tp_tp-10.rego` (package `shield_ai.tp.agent_platform_inbound_control`, 2 deny rules)
- **Incident evidence (13):** INC-24, INC-30, INC-31, INC-32, INC-34, INC-39, INC-45, INC-54, INC-56, INC-60, INC-61, INC-79, INC-92

TP-11: Agent browser and content-rendering isolation

What to verify: (agent browser runtimes and any component that renders and interprets untrusted content — agentic browsers, document processors in AI assistants, rendered web pages — must preserve origin and content isolation: rendered content is classified as untrusted input and stripped of imperceptible instruction vectors before agent action; cross-origin privileged actions (token access, credential use, financial operations) are gated by the same approval and isolation rules as any other high-risk tool action; the browser/session layer is isolated from the agent's credential material; and document uploads are sanitised before assistant processing

- **Executable policy:** `shield_ai_tp_tp-11.rego` (package `shield_ai.tp.agent_browser_and_content_rend`, 3 deny rules)
- **Incident evidence (4):** INC-29, INC-30, INC-42, INC-92

TP-12: Tool-call provenance and dispatch verification

What to verify: (every tool execution must verify that the tool-call instruction originated from an authorised model turn or an explicitly authorised caller — the

dispatch layer must not treat data shaped like a model-generated tool call as authoritative without provenance verification: tool calls carry cryptographically bound provenance (session, turn, model signature) that is verified before dispatch; session history and event streams are authenticated so that attacker-injected events cannot fabricate tool calls or approval confirmations; approval records bind to the exact tool call (tool name and arguments) they authorise and are verified against the human approver channel, never against session history the agent can write; and the inspection-execution gap fails closed — no provenance, no dispatch

- **Executable policy:** `shield_ai_tp_tp-12.rego` (package `shield_ai.tp.tool_call_provenance_and_dispa`, 4 deny rules)
- **Incident evidence (8):** INC-11, INC-34, INC-41, INC-44, INC-46, INC-53, INC-84, INC-92

TP-13: Agent dependency-install provenance verification

What to verify: (agents must not execute installation commands derived from ingested content — documentation pages, llms.txt/llms-full.txt files, setup guides, README instructions — without verifying the provenance of every package, domain, and command they reference: package names are confirmed to exist in the intended registry and the namespace is verified to belong to the documented vendor (registry ownership checks, domain registration and expiry checks) before any pip/npm/npx/ apt/go install or clone executes; installation commands sourced from untrusted or ingested content are gated by an operator-maintained allowlist or human approval instead of being treated as authoritative setup steps; dangling references — commands pointing to unregistered packages, unclaimed or expired domains, or namespaces that do not match the publisher — are treated as security findings that block the install, log via MIR-01, and surface for review; and a documentation file is never treated as an authoritative install script, because the file's perceived authority is exactly what the attack exploits

- **Executable policy:** `shield_ai_tp_tp-13.rego` (package `shield_ai.tp.agent_dependency_install_provenance`, 6 deny rules)
- **Incident evidence (7):** INC-44, INC-48, INC-66, INC-80, INC-85, INC-86, INC-92

TP-14: Skill/plugin payload integrity and instruction-channel isolation

What to verify: (skills, plugins, and marketplace extensions must be distributed as cryptographically signed, immutable payloads whose instruction content is fixed at install time; runtime-fetchable instruction channels — remote referral manifests, update feeds, and operator-controlled configuration fetched after install — are prohibited and must be blocked at both the network and runtime layers, because a post-install fetch channel is a command-and-control path invisible to MIR-01 (it is not a tool call); skills execute with privileges scoped to their declared capability set, never the agent's full identity, and a post-install behavioural baseline must be established under MIR-02 before full capability is granted, with baseline drift quarantining the skill; marketplace provenance must be attestable end-to-end (publisher → registry → artifact) with publisher identity attestation; on any signature, provenance, or baseline error the install is blocked and installed skills are suspended, and runtime-fetch attempts terminate the skill and alert via MIR-01 — never unsigned execution

- **Executable policy:** `shield_ai_tp_tp-14.rego` (package `shield_ai.tp.skill_plugin_payload_integrity`, 6 deny rules)
- **Incident evidence (3):** INC-48, INC-66, INC-92

TP-15: In-process code-execution sandbox sufficiency

What to verify: (denylist- and blocklist-based in-process code-execution sandboxes — code interpreters that confine LLM/agent-generated source by enumerating forbidden modules, attributes, calls, or string constants within the same process — must be treated as code-execution surfaces, not security boundaries: this class of sandbox is structurally bypassable because each denied primitive is one forgotten object-graph edge away from recovery of the full host context, and adversarial capability escalation re-derives arbitrary execution from any surviving reachable built-in or bridge; therefore any LLM-generated or untrusted code destined for in-process execution must instead run inside an OS- or VM-level isolation boundary (containers, VMs, gVisor-style runtimes, or dedicated subprocesses with kernel-enforced separation — see EE-01) with no shared host filesystem or IPC, and in-process interpreter gates must not be deployed as the sole containment for untrusted code; where an in-process sandbox is unavoidable in the tooling stack, it must be treated as the HIGH-RISK dependency class of TP-06 with automated blocking on any critical finding and a mandatory alternative-evaluation gate for any L3+ deployment;

the in-process rule extends beyond denylists to any same-process isolation scheme whose security boundary relies on separating untrusted from trusted code by an in-process mechanism — separate language contexts or virtual machines within a single host process are not security boundaries when they share a memory heap or when any trust secret (an authorisation token separating trusted from untrusted code) lives in memory that the untrusted side can read (the Codex Heapjack flaw, INC-65, Accomplish AI, disclosed September 15, 2026: `node_repl` ran trusted and untrusted JavaScript in two V8 vm contexts within a single Node process sharing one heap, and the untrusted context recovered the trusted-authorisation token from a heap snapshot with `v8.getHeapSnapshot()`, then injected its own requests onto the same pipe the trusted context used to reach the native, unsandboxed parent process — host command execution with no approval from Codex's strictest read-only mode), so in-process trusted/untrusted separation must be replaced by process-, container-, or VM-level isolation where untrusted code communicates across an OS-enforced boundary and any authorisation secret is held outside the untrusted memory space

- **Executable policy:** `shield_ai_tp_tp-15.rego` (package `shield_ai.tp.in_process_code_execution_sandbox_sufficiency`, 5 deny rules)
- **Incident evidence (11):** INC-58, INC-65, INC-68, INC-74, INC-76, INC-77, INC-80, INC-85, INC-87, INC-91, INC-92

TP-16: Tool-connection and launch-path integrity

What to verify: (agent/tool connection adapters must treat remote peer-supplied protocol metadata — OAuth discovery documents, authorization_endpoint values, server definitions, connect-time responses, and any string an external server controls — as untrusted input that must never reach an OS-process launch, shell, or browser-launch primitive directly: an adapter that hands a server-supplied string to a process launcher (browser open, shell exec, PowerShell EncodedCommand, IDE launch) is a code-execution boundary regardless of URL-scheme checks, because a syntactically valid HTTPS URL can still carry a payload the launcher evaluates (PowerShell $(…) subexpression expansion in double-quoted strings); the authorization endpoint must be origin-bound to the configured server and validated on the canonical processed value after any encoding, never accepted on the server's advertised value as authoritative without comparison against the configured origin; tool/connection onboarding and reconnection must assume a later compromise of a previously

benign server, a hijacked registry listing, a typosquatted name, or an expired domain are in-threat-model, because the payload arrives at connection time from a peer whose hostname looked legitimate; launch commands registered through tool or server definitions are validated and allowlisted, and a model-influenced or externally-derived server definition must not be able to register an arbitrary OS command an IDE or agent will auto-launch; fail closed: any transport field crossing into an execution/launch primitive terminates the connection and alerts (MIR-01)

- **Executable policy:** `shield_ai_tp_tp-16.rego` (package `shield_ai.tp.tool_connection_and_launch_path_integrity`, 6 deny rules)
- **Incident evidence (6):** INC-79, INC-85, INC-86, INC-88, INC-89, INC-92

MIR — Monitoring, Logging & IR

MIR-01: Agent activity logging

What to verify: all agent actions logged – every tool invocation, data access, decision, and error; logs include agent identity, session ID, tool, input summary, output summary

- **Executable policy:** `shield_ai_mir_mir-01.rego` (package `shield_ai.mir.agent_activity_logging`, 2 deny rules)
- **Incident evidence (10):** INC-01, INC-05, INC-06, INC-08, INC-12, INC-16, INC-19, INC-48, INC-56, INC-92

MIR-02: Behavioural anomaly detection baseline per-agent behaviour established over training period; alerts on deviation – unusual tool, unusual data access pattern, unusual frequency;

What to verify: FOFA-style), mass exploitation of exposed instances, and self-propagating campaign patterns, as demonstrated by the Unit 42 campaign where a DeepSeek-driven agent autonomously scanned 84 exposed Langflow instances from a single operator task, and by Kimi K3 agents autonomously developing Redis exploit chains — plus unexpected listener/binding events from agent environments; must also include agent memory-store tamper detection — poisoned pattern/learning stores steer future outputs and survive code patches (Ruflo/RufRoot CVE-2026-59726) — and detection of agent-caused changes to third-party systems

outside the organisation's boundary, as when Meta's Muse Spark breached an external company during a misconfigured evaluation and a consumer OpenClaw agent autonomously exploited a gym-booking API to cancel another user's reservation; MIR-02 must additionally detect agentic intrusions in which human-directed agent-CLI use is the intrusion platform — cross-session command clustering, guardrail-erosion prompting patterns (bug-bounty framing, escalating privilege requests, long playbook instructions), credential-touch patterns, and low-frequency long-dwell activity; detection must trigger automated first response via pre-approved containment playbooks and must activate the exercised GRM-02 kill-switch, and detection-pipeline health is itself monitored so an outage cannot silently blind this control — on detection-pipeline error or outage, high-risk agent execution suspends to human-only mode, and kill-switch activation must return verifiable acknowledgment or the incident auto-escalates to human responders;

- **Executable policy:** `shield_ai_mir_mir-02.rego` (package `shield_ai.mir.behavioural_anomaly_detection`, 2 deny rules)
- **Incident evidence (18):** INC-06, INC-08, INC-10, INC-19, INC-23, INC-27, INC-28, INC-32, INC-35, INC-40, INC-43, INC-45, INC-49, INC-55, INC-56, INC-63, INC-90, INC-92

MIR-03: Prompt audit log

What to verify: capture sanitised prompt/response pairs for forensic analysis; sensitive fields redacted before logging but recoverable for incident response

- **Executable policy:** `shield_ai_mir_mir-03.rego` (package `shield_ai.mir.prompt_audit_log`, 2 deny rules)
- **Incident evidence (1):** INC-22

MIR-04: Incident response playbooks AI-specific IR procedures distinct from general IR: prompt injection containment, model rollback, tool access revocation, session invalidation;

What to verify: the Hugging Face defensive-asymmetry lesson from the OpenAI-Hugging Face intrusion, where responders had to deploy a locally hosted open-

weight model after commercial models refused to analyse the attacker's exploit code); IR playbooks must define

- **Executable policy:** `shield_ai_mir_mir-04.rego` (package `shield_ai.mir.incident_response_playbooks`, 2 deny rules)
- **Incident evidence (3):** INC-07, INC-16, INC-92

MIR-05: Mandatory reporting integration

What to verify: automated triggers for SOCI 12-hour and APRA 72-hour notification windows; pre-populated incident report templates

- **Executable policy:** `shield_ai_mir_mir-05.rego` (package `shield_ai.mir.mandatory_reporting_integratio`, 1 deny rules)
- **Incident evidence:** none yet — baseline coverage, not yet demonstrated by a catalogue incident

MIR-06: Forensic readiness

What to verify: immutable logs written to append-only storage; full session reconstruction capability including tool call chains and model responses

- **Executable policy:** `shield_ai_mir_mir-06.rego` (package `shield_ai.mir.forensic_readiness`, 1 deny rules)
- **Incident evidence (6):** INC-07, INC-40, INC-46, INC-53, INC-70, INC-92

GRM — Governance & Risk

GRM-01: AI governance body

What to verify: chartered committee with CISO, CRO, General Counsel, data owner, and AI lead; terms of reference that include agentic AI approval authority)

- **Executable policy:** `shield_ai_grm_grm-01.rego` (package `shield_ai.grm.ai_governance_body`, 3 deny rules)
- **Incident evidence (1):** INC-92

GRM-02: Requirement:** AI risk register

What to verify: AI risk register (distinct from general risk register; AI-specific threat taxonomy covering model, tool, data, identity, and pipeline risks;

- **Executable policy:** `shield_ai_grm_grm-02.rego` (package `shield_ai.grm.ai_risk_register`, 2 deny rules)
- **Incident evidence (6):** INC-06, INC-32, INC-35, INC-49, INC-55, INC-92

GRM-03: Ethical use policy

What to verify: acceptable use boundaries for agentic AI: what agents can and cannot do; human-in-the-loop requirements by risk tier; transparency obligations)

- **Executable policy:** `shield_ai_grm_grm-03.rego` (package `shield_ai.grm.ethical_use_policy`, 2 deny rules)
- **Incident evidence (3):** INC-10, INC-32, INC-42

GRM-04: Requirement:** Third-party AI risk management

What to verify: Third-party AI risk management (vendor security assessment for model providers, tool providers, and infrastructure; contractual obligations for security controls; right-to-audit clauses; incident-response planning with AI/agent vendors must include agent-speed adversary playbooks and automatic containment,

- **Executable policy:** `shield_ai_grm_grm-04.rego` (package `shield_ai.grm.third_party_ai_risk_management`, 3 deny rules)
- **Incident evidence (6):** INC-26, INC-62, INC-63, INC-75, INC-90, INC-92

GRM-05: Regulatory mapping and compliance reporting

What to verify: automated mapping to ISM, E8, SOCI, CPS 234, PSPF, Privacy Act; quarterly compliance posture reports)

- **Executable policy:** `shield_ai_grm_grm-05.rego` (package `shield_ai.grm.regulatory_mapping_and_complia`, 2 deny rules)
- **Incident evidence (1):** INC-16

GRM-06: Requirement:** Board and executive reporting

What to verify: Board and executive reporting (standardised dashboard for AI security posture: maturity scores per pillar, open findings, regulatory compliance status, risk acceptance decisions;

- **Executable policy:** `shield_ai_grm_grm-06.rego` (package `shield_ai.grm.board_and_executive_reporting`, 2 deny rules)
- **Incident evidence (3):** INC-73, INC-90, INC-92

Verification protocol

- Run `make test` — the OPA suite evaluates every `policies/*.rego` (69 files, 195 deny rules).
- A control is met when its policy's `deny` set is empty for the input under test.
- Missing input fields are treated as violations (fail closed).